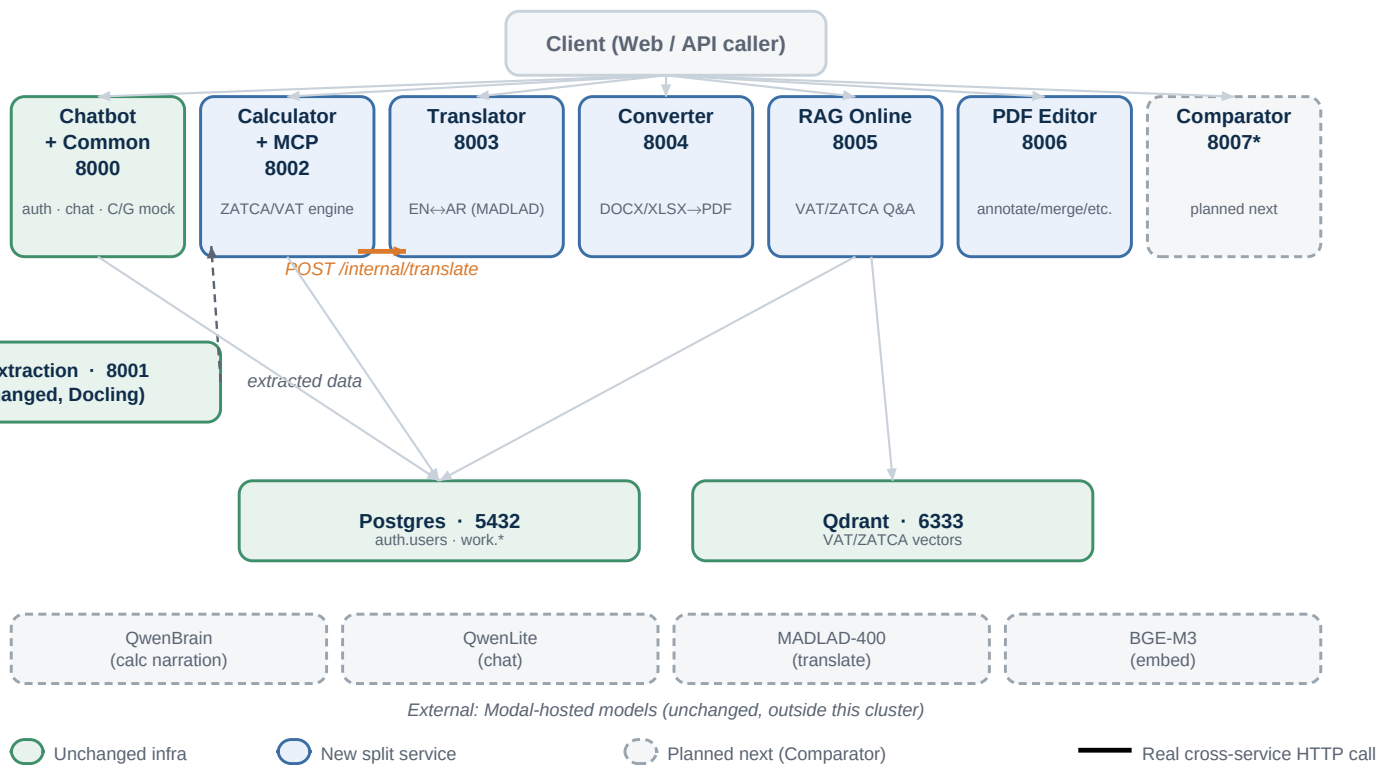


Mizan.ai — Microservices Architecture

From a single modular monolith to 7 independently deployable services, designed for a Kubernetes deployment on GCP.

System Overview



Every service verifies JWTs independently against a shared secret (stateless auth — only Chatbot + Common mints tokens). Postgres and Qdrant are shared, independently-run infrastructure, not owned by any one service. The one deliberate synchronous service-to-service call in the whole system is Calculator → Translator, used only when a compliance report needs a translated line-item description.

Why This Is a Real Microservices Architecture

The starting point was a modular monolith: one FastAPI process, well-organized internally by feature folder, but deployed as a single unit bundling seven distinct business capabilities. Splitting code into folders is not, by itself, a microservices architecture — what makes a split real is that each resulting unit can be deployed, scaled, and taken down independently, and that dependencies between units cross a genuine network/API boundary rather than a shared Python import.

Every merge/split decision in this system was made against one explicit test, applied consistently rather than bundling services by convenience:

“Does this piece need to scale independently, or does it need to be called by more than one other service?”

Applying that test led to decisions that cut both ways — merging where a split would add nothing, and keeping services separate even when it looked like convenient bundling:

- **Calculator + MCP were merged** into one service. The MCP tool server has exactly one caller (Calculator itself); splitting it into its own container would add a network hop with zero benefit. This gets revisited only if a second consumer of the same tools appears — e.g. if the future Comparator service ever needs the same report-rendering tools, in which case the underlying code gets shared as a library first, not defaulted into a live cross-service call.
- **Translator and Converter were kept separate** despite both being small. They share no code and have opposite resource profiles: Translator is a thin proxy to a hosted model, Converter runs a real LibreOffice subprocess with its own CPU/concurrency cost. Bundling them would force unrelated load patterns to scale together.
- **RAG Online was split out** specifically for its Qdrant dependency and retrieval-heavy load profile, distinct from general chat.
- **PDF Editor was kept separate** from Converter ahead of a planned v2 that adds heavier features to it — isolating the boundary now avoids a disruptive split later.
- **Comparator (Feature C) was deliberately not merged** into Calculator despite overlapping report-generation logic — different business domain (reconciliation vs. tax calculation), different scaling trigger. The two will share report-rendering code, not a deployable unit, once Comparator is built.

Service Catalog

Service	Port	Owns	Depends on
Chatbot + Common	8000	Auth (JWT mint/verify), general chat, mock Comparator/Synthesis endpoints	Postgres
Calculator + MCP	8002	ZATCA VAT engine, structural validation, classification, report generation	Postgres, Translator (HTTP), QwenBrain
Translator	8003	EN↔AR translation (public + internal endpoint)	MADLAD-400 (Modal)
Converter	8004	DOCX/XLSX → PDF via LibreOffice	— (local subprocess)
RAG Online	8005	Retrieval-grounded VAT/ZATCA Q&A	Postgres, Qdrant, BGE-M3, QwenLite
PDF Editor	8006	Annotate, merge, split, redact, watermark, etc.	— (local, PyMuPDF)
doc-extraction	8001	Document text/field extraction (unchanged)	— (local, Docling)

Real Boundaries, Not Just Folders

Three concrete in-process couplings had to be resolved before the split was real rather than cosmetic — each one is exactly the kind of dependency that silently defeats a “microservices” split if left in place:

- **Calculator → Translator.** Report generation used to import `translate_paragraph()` directly from the Translator module. Now it makes a real HTTP call to Translator's `POST /internal/translate` endpoint — unauthenticated by design (never called by end users directly, same pattern already used by doc-extraction), and fails open: a translation failure degrades to the original text rather than breaking report generation.
- **Shared language detection.** `detect_language()` used to live inside the Chatbot module and was imported by three other features. It moved to a genuinely shared `agents/common/` module so Translator, RAG Online, and Calculator no longer depend on Chatbot's package at all.
- **Cross-service data ownership.** Calculator's filing-notes Q&A endpoint reads `work.user_filing_notes`, a table whose schema is owned by Chatbot + Common. Rather than adding a synchronous HTTP dependency for a single-consumer read, Calculator reads Postgres directly — consistent with the same boundary test above: Postgres is already the independently-run, shared component, and there is only one consumer of this read. Schema creation itself is idempotent (`CREATE ... IF NOT EXISTS` throughout) and is called independently by both services at their own startup, so neither one depends on the other's startup order — verified directly by starting Calculator with Chatbot + Common stopped.

Deliberately Not Split

Not every boundary needs to move. These stayed exactly as they were, because splitting them would add operational cost without a corresponding scaling or reuse need:

- **Postgres and Qdrant** were already independent, shared containers — nothing to split further.
- **doc-extraction** was already its own service (isolated for Docling's heavy dependency chain, unrelated to this pass).
- **RAG offline ingestion** stays a manual/periodic job, not a live service — it's a batch workload, not a request-serving one, and belongs to a different Kubernetes workload type (Job/CronJob) when it eventually moves there.
- **Modal-hosted models** (QwenBrain, QwenLite, MADLAD-400, BGE-M3) are external to this cluster entirely.

What's Next

Comparator (document reconciliation) is the next feature to be built, and will ship as its own independent service (`services/comparator/`), completing the 7-service split — not folded into Calculator despite the report-generation overlap, per the same boundary reasoning used throughout this split. Beyond that, this docker-compose setup is the deliberate proving ground for the target production deployment: a GCP Compute Instance running these same services as Kubernetes Deployments, with each service's own Docker image and port mapping directly translating into its own Deployment + Service manifest.